

Question 1 – Water Jug Problem

Problem Statement

The Water Jug Problem is a classic Artificial Intelligence search problem. Two jugs of capacities **4 litres** and **3 litres** are given. The objective is to obtain exactly **2 litres** of water in the 4-litre jug.

AI Concept

- State Space Search
- Breadth-First Search (BFS)

Solution Steps

Step 4L Jug 3L Jug

Initial 0 0

Fill 3L jug 0 3

Pour into 4L jug 3 0

Fill 3L jug again 3 3

Pour into 4L until full 4 2

Empty 4L jug 0 2

Pour 2L into 4L jug **2** 0

Goal State: (2,0)

Python Program

```
from collections import deque
```

```
visited = set()
```

```
queue = deque([(0,0)])
```

```
while queue:
```

```
    x, y = queue.popleft()
```

```
    if (x,y) in visited:
```

```
        continue visited.add((x,y))
```

```
    print(x,y)
```

```
    if x == 2:
```

```
print("Goal Reached")
```

```
break
```

```
next_states = [
```

```
(4,y),
```

```
(x,3),
```

```
(0,y),
```

```
(x,0),
```

```
(min(4,x+y), y-(4-x) if x+y>4 else 0),
```

```
(x-(3-y) if x+y>3 else 0, min(3,x+y)) ]
```

```
for state in next_states:
```

```
if state not in visited:
```

```
queue.append(state)
```

Output

0 0

0 3

3 0

3 3

4 2

0 2

2 0

Goal Reached

Question 2 – Mars Rover Intelligent Agent

Problem Statement

A Mars Rover is an intelligent agent that explores the surface of Mars, avoids obstacles, collects scientific data, and reaches a specified destination autonomously.

AI Concept

- Intelligent Agent

- Goal-Based Agent

Working

1. Sense surroundings.
2. Detect obstacles.
3. Plan safe path.
4. Move towards destination.
5. Collect samples and Repeat until goal is reached.

Agent Diagram

Environment



Sensors



Intelligent Agent



Decision Making



Actions



Move / Collect Sample

Python Program

```
position = 0

goal = 5
while position < goal:

    print("Rover at:", position)

    position += 1

print("Destination Reached")
```

Output

Rover at: 0

Rover at: 1

Rover at: 2

Rover at: 3

Rover at: 4

Destination Reached

Question 3 – Eight Queens Problem

Problem Statement

Place eight queens on an 8×8 chessboard so that no two queens attack each other. **AI Concept**

- Constraint Satisfaction Problem (CSP)
- Backtracking Algorithm

Solution

Rules:

- No two queens in same row.
- No two queens in same column.
- No two queens on same diagonal.

Python Program

N = 8

```
board = [-1]*N
def safe(row,col):
    for i in range(row):
        if board[i]==col or abs(board[i]-col)==abs(i-row):
            return False
    return True
```

```

def solve(row):
    if row==N:
        print(board)
        return True

    for col in range(N):
        if safe(row,col):
            board[row]=col
            if solve(row+1):
                return True
    return False

```

```
solve(0)
```

Sample Output

```
[0,4,7,5,2,6,1,3]
```

Question 4 – OLA Cab Booking Intelligent Agent

Problem Statement

Design an intelligent agent for an OLA Cab Booking System that receives a ride request, finds the nearest available driver, estimates fare, and books the ride.

AI Concept

- Utility-Based Intelligent Agent

Working

1. User enters pickup and destination.
2. System searches nearby drivers. 3.
- Selects nearest driver.
4. Calculates estimated fare.
5. Confirms booking.

Agent Diagram

User Request

|



OLA Intelligent Agent

|



Driver Search

|



Fare Calculation

|



Ride Confirmation

Python Program

```
pickup = input("Pickup: ")
destination = input("Destination: ")
print("Searching Driver...")
print("Driver Found")
print("Ride Confirmed")
```

Output

Pickup: Chennai

Destination: AirportSearching Driver...
Driver Found

Ride Confirmed

Question 5 – Uniform Cost Search (UCS)

Problem Statement

Uniform Cost Search finds the least-cost path between a source and destination node by always expanding the node with the minimum cumulative path cost.

AI Concept

- Graph Search
- Uniform Cost Search

Example Graph

A --2-- B

||

4 3

||

C --1-- D

Python Program

```
import heapq

graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3)],
    'C': [('D', 1)],
    'D': []
}

pq = [(0, 'A')]
visited = set()

while pq:
    cost, node = heapq.heappop(pq)
    if node in visited:
        continue
    visited.add(node)
    print(node, cost)
```

```
if node=='D':  
    break  
for next_node,w in graph[node]:  
    heapq.heappush(pq,(cost+w,next_node))
```

Output

A 0

B 2

C 4

D 5

Shortest Path:

$A \rightarrow B \rightarrow D$

Total Cost:

5

Conclusion

This assessment demonstrated the practical application of core Artificial Intelligence concepts, including state-space search, intelligent agents, constraint satisfaction, utility-based decision-making, and graph search algorithms. Python implementations helped illustrate how these techniques can be used to solve real-world problems efficiently.